

Présentation

Le but de ce TP est de découvrir le parallélisme à travers la création de *processus* et de *threads* sous UNIX. Cette possibilité offerte par le système se révèle utile lorsqu'on cherche à implanter des algorithmes parallèles. On se propose de paralléliser l'algorithme de tri connu qui est qualifié d'algorithme de tri le plus efficace. Son but est d'ordonner un vecteur v de manière croissante de $v[0]$ jusqu'à $v[n-1]$, où n est la taille du vecteur.

Tri mono processus

La première version du programme *tri_rapide* se base sur une méthode qui utilise un seul processus, par un appel d'une fonction récursive *tri* (). La méthode à utiliser commence par sélectionner un élément appelé p_v d'indice i_p parmi les n du vecteur $v[]$. On retient dans ce TP l'élément du milieu du vecteur comme l'élément p_v . On permute alors les éléments du tableau selon une méthode qui permet de mettre tous les éléments dont la valeur est inférieure à p_v dans $v[0]$ jusqu'à $v[x]$ et tous ceux dont la valeur est supérieure dans $v[x+2]$ à $v[n-1]$. Donc l'élément p_v se retrouvera forcément à l'indice $x+1$.

On reprend ensuite le tri récursivement pour les deux sous tableaux de $v[0]$ à $v[x]$ et puis de $v[x+2]$ à $v[n-1]$. Comme tous les éléments du segment de droite sont inférieurs ou égales à ceux du côté gauche, le tableau sera ainsi simplement trié.

Tri parallèle

Dans les appels récursifs précédents, les deux appels sur les sous tableaux sont logiquement indépendants. Ils peuvent donc être réalisés par deux processus différents, ce qui fournit un schéma de parallélisation de notre fonction de tri. Donc, pour réaliser le travail demandé, le processus père crée deux processus fils dont chacun se charge d'un sous tableau, attend leurs terminaisons, imprime la partie du tableau résultat qui lui revient et conclue. Le même traitement est réalisé récursivement à chaque fois au niveau des deux parties.

Tri parallèle à l'aide des threads

On s'intéresse maintenant à l'utilisation des threads au lieu des processus de la deuxième étape. Comme les threads partagent le même espace de travail que le processus père, c'est au père alors d'imprimer le tableau à la fin du tri.

Travail demandé

Réaliser ce travail pratique en passant par les trois étapes indiquées ci-dessus. Le système doit comprendre une interface (*textuelle* seulement, sous forme d'impressions) permettant de suivre et de comprendre le fonctionnement des différentes entités du service objet de ce TP.

Connaissances requises

- Langage de programmation C.
- Appels systèmes : création de processus/threads, attente de la fin d'un processus/thread.

Echéance

29 Octobre 2024.